Relational Ability Experiments With Tabular Foundation Models

The experiments compare:

- Tabular foundation models: `TabPFN`, `TabICL`.
- Knowledge-graph embedding baselines: `DistMult`, `ComplEx`, `TransE`, `TransR`.

The central question is:

```
Can tabular foundation models infer relational properties from tabular in-
context evidence over entity pairs?
```

The experiments evolved through four datasets:

- group-based symmetry;
- non-transitive symmetry;
- total-order antisymmetry;
- sparse `supervised_by` antisymmetry.

The most important final result is the sparse `supervised_by` experiment. In that realistic sparse antisymmetry setting, `TabICL` is stronger than the implemented KGE baselines by F1, average precision, and ROC AUC.

Shared Experimental Interface

All model-facing datasets use the same schema:

```
entity_1,entity_2,label,split
```

The model receives only:

```
entity_1
entity_2
```

The target is:

```
label
```

The split column is used only by the runner to select train, validation, and test rows. It is not passed as a model feature.

Hidden variables such as group ID, seniority rank, role, student identity, or supervisor identity are written only to audit files. They are not written to the model-facing CSV.

The main runner validates this in `run_minimal_symmetry_tfm_experiment.py`:

- `load_dataset_rows()` reads the CSV and requires `entity_1`, `entity_2`, `label`, `split`.
- It rejects obvious leakage columns such as `hidden_group_id`.
- It checks that train and test contain both classes.

Code reference:

```
run_minimal_symmetry_tfm_experiment.py:145
```

The KGE runner performs the same kind of validation:

```
run_kge_relation_baselines.py:142
```

Train, Validation, And Test Are Not Merged

The TFM runner partitions the data by the `split` column:

```
train_df = rows where split == train
valid_df = rows where split == validation
test_df  = rows where split == test
```

Code reference:

```
run_minimal_symmetry_tfm_experiment.py:525
```

Models are fit only on train:

```
model.fit(X_train_in, y_train)
```

Code reference:

```
run_minimal_symmetry_tfm_experiment.py:447
```

Validation is used only for threshold tuning when:

```
--threshold-mode tune_validation_f1
```

The final metrics are always computed on test:

```
run_minimal_symmetry_tfm_experiment.py:550
```

The KGE runner uses the same split logic:

```
run_kge_relation_baselines.py:503
run_kge_relation_baselines.py:520
run_kge_relation_baselines.py:539
```

Model Computation In The Code

TFM Computation

TFM construction happens here:

```
run_minimal_symmetry_tfm_experiment.py:202
run_minimal_symmetry_tfm_experiment.py:219
```

`TabPFN` receives integer categorical codes:

```
run_minimal_symmetry_tfm_experiment.py:193
```

Both columns are declared categorical for TabPFN:

```
categorical_features_indices=[0, 1]
```

`TabICL` receives pandas categorical columns directly:

```
run_minimal_symmetry_tfm_experiment.py:439
```

The actual fit/predict computation is:

```
run_minimal_symmetry_tfm_experiment.py:447
run_minimal_symmetry_tfm_experiment.py:452
```

```
run_minimal_symmetry_tfm_experiment.py:453
```

Threshold tuning is:

```
run_minimal_symmetry_tfm_experiment.py:251
```

Metric computation is:

```
run_minimal_symmetry_tfm_experiment.py:265
```

KGE Baseline Computation

The KGE runner implements supervised binary versions of the scoring functions.

The model definitions are in:

```
run_kge_relation_baselines.py:229
```

The implemented scores are:

```
DistMult: score(A, B) = sum(e_A * r * e_B) + bias
ComplEx: complex-valued bilinear score
TransE:   score(A, B) = scale * (margin - distance(e_A + r, e_B)) + bias
TransR:   score(A, B) = scale * (margin - distance(M e_A + r, M e_B)) +
bias
```

Important code references:

```
DistMult forward: run_kge_relation_baselines.py:239
ComplEx forward:  run_kge_relation_baselines.py:257
TransE forward:   run_kge_relation_baselines.py:282
TransR forward:   run_kge_relation_baselines.py:310
```

Training uses explicit binary labels and `BCEWithLogitsLoss`:

```
run_kge_relation_baselines.py:359
run_kge_relation_baselines.py:361
run_kge_relation_baselines.py:382
run_kge_relation_baselines.py:383
```

This is not a full canonical link-prediction training setup with corrupted triples and ranking loss. It is a supervised binary classification adaptation of the KGE scoring functions to our tabular datasets.

Validation loss is used for early stopping:

```
run_kge_relation_baselines.py:389
run_kge_relation_baselines.py:396
```

Validation probabilities are also used for optional F1 threshold tuning:

```
run_kge_relation_baselines.py:197
run_kge_relation_baselines.py:419
```

Test metrics are computed here:

```
run_kge_relation_baselines.py:539
```

Experiment 1: Group-Based Symmetry

Dataset

Generator:

```
generate_minimal_symmetry_dataset.py
```

The hidden world is made of groups.

The label rule is:

```
label(A, B) = 1 if A and B are in the same hidden group
label(A, B) = 0 otherwise
```

Code reference:

```
generate_minimal_symmetry_dataset.py:136
generate_minimal_symmetry_dataset.py:146
```

The split is created by exporting both directions of each selected unordered pair:

```
generate_minimal_symmetry_dataset.py:204
generate_minimal_symmetry_dataset.py:228
generate_minimal_symmetry_dataset.py:239
```

In `symmetry_demonstration` mode, demo pairs have both directions in train. Query pairs have one direction in train and the reverse direction in validation/test.

Dataset metadata:

```
entities: 60
hidden groups: 12
group size: 5
atom rows: 480
train rows: 360
validation rows: 24
test rows: 96
label balance: 240 positives, 240 negatives
split mode: symmetry_demonstration
```

Important Methodological Note

This dataset is symmetric, but it is also transitive.

If:

```
R(A, B) = 1
R(B, C) = 1
```

then all three entities are in the same hidden group, so:

```
R(A, C) = 1
```

Therefore, the task is not only symmetry. It is closer to an equivalence-relation or clustering task.

Results

TFM results:

model	threshold	accuracy	F1	precision	recall	AP	ROC AUC
TabPFN	0.500	0.427	0.530	0.443	0.660	0.419	0.366
TabICL	0.500	0.438	0.413	0.422	0.404	0.417	0.380

model	threshold	accuracy	F1	precision	recall	AP	ROC AUC
DistMult baseline in TFM runner	0.500	1.000	1.000	1.000	1.000	1.000	1.000

KGE results:

model	threshold	accuracy	F1	precision	recall	AP	ROC AUC
ComplEx	0.500	0.500	0.547	0.492	0.617	0.539	0.535
TransR	0.500	1.000	1.000	1.000	1.000	1.000	1.000
TransE	0.500	1.000	1.000	1.000	1.000	1.000	1.000
DistMult	0.500	1.000	1.000	1.000	1.000	1.000	1.000

Interpretation

The dataset is clearly learnable by models with relational/geometric inductive bias.

`DistMult` succeeds because its score is symmetric by construction:

```
score(A, B) = score(B, A)
```

`TransE` and `TransR` succeed because the group structure can be represented geometrically: same-group entities are placed close together, different-group entities far apart. For a symmetric relation, TransE can learn relation vector close to zero, reducing the task to distance between entity embeddings.

The TFMs do not exploit this structure from the raw categorical pair table.

The result supports:

```
The task is solvable, but TabPFN and TabICL do not recover the group-based
symmetry structure from this representation.
```

The result does not support:

```
The task is a pure test of abstract symmetry.
```

because the dataset also contains transitive cluster structure.

Experiment 2: Non-Transitive Symmetry

Dataset

Generator:

```
generate_nontransitive_symmetry_dataset.py
```

This dataset was added to remove the transitive group shortcut.

The hidden relation is a random undirected graph:

```
label(A, B) = 1 if unordered edge {A, B} was sampled
label(A, B) = 0 otherwise
```

The relation is symmetric because every selected unordered pair is exported in both directions with the same label.

The relation is non-transitive because positive edges are not closed into cliques.

Code references:

```
sample positive graph edges: generate_nontransitive_symmetry_dataset.py:106
count transitivity violations:
generate_nontransitive_symmetry_dataset.py:130
assign labels from graph edges:
generate_nontransitive_symmetry_dataset.py:158
assign reverse-holdout split:
generate_nontransitive_symmetry_dataset.py:217
validate symmetry and non-transitivity:
generate_nontransitive_symmetry_dataset.py:289
```

Dataset metadata:

```
entities: 80
positive unordered edges: 160
atom rows: 640
train rows: 320
validation rows: 64
test rows: 256
label balance: 320 positives, 320 negatives
split mode: reverse_holdout
transitivity violations: 592
```

A transitivity violation means:

```
R(A, B) = 1
R(B, C) = 1
```

```
R(A, C) = 0
```

The generator explicitly verifies that such violations exist.

Results

KGE results:

model	threshold	accuracy	F1	precision	recall	AP	ROC AUC
ComplEx	0.500	0.523	0.470	0.519	0.429	0.548	0.541
TransR	0.500	1.000	1.000	1.000	1.000	1.000	1.000
TransE	0.500	1.000	1.000	1.000	1.000	1.000	1.000
DistMult	0.500	1.000	1.000	1.000	1.000	1.000	1.000

No non-transitive TFM result folder is present in the current workspace. The available result for this dataset is the KGE result.

Interpretation

This experiment removes the transitive cluster shortcut, but it still uses `reverse_holdout`.

In `reverse_holdout`, every test row is the reverse of a pair already seen in train:

```
train: (A, B), label
test:  (B, A), same label
```

This is a direct symmetry test. It is also exactly the kind of task where symmetric KGE scoring functions should be strong.

`DistMult` must succeed if it learns the train pair, because:

```
score(A, B) = score(B, A)
```

`TransE` and `TransR` can also succeed by learning a relation vector close to zero:

```
score(A, B) approximately -distance(e_A, e_B)
score(B, A) approximately -distance(e_B, e_A)
```

and distance is symmetric.

Therefore, the 100 percent result does not mean TransE/TransR learned an abstract symmetry rule. It means the reverse-holdout protocol is naturally solved by symmetric or near-symmetric scoring geometry.

The important methodological result is:

```
Non-transitivity was successfully introduced, but reverse-holdout remains
easy for KGE baselines.
```

Experiment 3: Total-Order Antisymmetry

Dataset

Generator:

```
generate_minimal_antisymmetry_dataset.py
```

The relation is:

```
senior_coauthor_of
```

Each entity receives a hidden seniority rank:

```
rank 1, rank 2, ..., rank N
```

Code reference:

```
generate_minimal_antisymmetry_dataset.py:79
generate_minimal_antisymmetry_dataset.py:88
```

The label rule is:

```
label(A, B) = 1 if rank(A) < rank(B)
label(A, B) = 0 otherwise
```

Code reference:

```
generate_minimal_antisymmetry_dataset.py:161
```

The split exports one direction to train and the opposite direction to train/validation/test depending on pair role:

```
generate_minimal_antisymmetry_dataset.py:172
generate_minimal_antisymmetry_dataset.py:197
generate_minimal_antisymmetry_dataset.py:208
```

Dataset metadata:

```
entities: 60
unordered pairs sampled: 240
atom rows: 480
train rows: 360
validation rows: 24
test rows: 96
label balance: 240 positives, 240 negatives
split mode: antisymmetry_demonstration
```

Important Methodological Note

This dataset is antisymmetric, but it is also highly structured.

It is a total order:

```
A is more senior than B
B is more senior than C
therefore A is more senior than C
```

This is easier than a generic antisymmetric relation. It is especially easy for TransE/TransR because a global order can be represented as a geometric direction.

Results

TFM results:

model	threshold	accuracy	F1	precision	recall	AP	ROC AUC
TabPFN	0.500	0.625	0.640	0.681	0.604	0.741	0.722
TabICL	0.500	0.635	0.653	0.688	0.623	0.734	0.728
DistMult baseline in TFM runner	0.500	0.323	0.000	0.000	0.000	0.367	0.085

KGE results:

model	threshold	accuracy	F1	precision	recall	AP	ROC AUC
ComplEx	0.500	0.521	0.511	0.585	0.453	0.591	0.517
TransR	0.500	0.979	0.981	1.000	0.962	0.993	0.987
TransE	0.500	0.979	0.981	1.000	0.962	0.998	0.997
DistMult	0.500	0.333	0.179	0.280	0.132	0.459	0.325

Interpretation

`DistMult` fails as expected because it is symmetric:

```
score(A, B) = score(B, A)
```

It cannot naturally represent:

```
R(A, B) = 1
R(B, A) = 0
```

`TransE` and `TransR` succeed because the dataset is not arbitrary antisymmetry. It is a global ranking relation. A translation model can represent a direction from less senior to more senior or vice versa.

This is consistent with the Trouillon et al. observation that TransE can struggle on arbitrary antisymmetric relations. Our total-order antisymmetry is easier than their arbitrary generated antisymmetric setting.

The TFMs are above chance but much weaker than TransE/TransR here.

The correct claim is:

```
TFMs show some directional signal on total-order antisymmetry, but KGE
translation models solve the structured ranking relation much better.
```

The incorrect claim would be:

```
TransE/TransR solve arbitrary antisymmetry.
```

This experiment does not test arbitrary antisymmetry. It tests a total-order variant.

Experiment 4: Sparse `supervised_by` Antisymmetry

Dataset

Generator:

```
generate_sparse_supervised_by_dataset.py
```

This is the most realistic antisymmetry experiment.

The relation is:

```
supervised_by(student, supervisor) = 1
supervised_by(supervisor, student) = 0
```

But unlike the total-order dataset, unrelated pairs are negative in both directions:

```
supervised_by(A, B) = 0
supervised_by(B, A) = 0
```

This makes the relation sparse and graph-like rather than a total ranking.

The generator creates hidden roles:

```
student
supervisor
```

but roles are not written to the model-facing CSV.

Code references:

```
create hidden roles: generate_sparse_supervised_by_dataset.py:77
sample supervision edges: generate_sparse_supervised_by_dataset.py:125
build positive/reverse-negative and unrelated both-negative pairs:
generate_sparse_supervised_by_dataset.py:151
assign train/validation/test split:
generate_sparse_supervised_by_dataset.py:252
validate sparse antisymmetry: generate_sparse_supervised_by_dataset.py:343
```

The important label logic is:

```
supervision edge:
    student -> supervisor = 1
    supervisor -> student = 0

unrelated pair:
```

```
    A -> B = 0
    B -> A = 0
```

Code reference:

```
generate_sparse_supervised_by_dataset.py:170
generate_sparse_supervised_by_dataset.py:195
```

Current result metadata:

```
train rows: 272
validation rows: 54
test rows: 218
threshold mode: tune_validation_f1
test positive rate: 0.1284
```

The test set is strongly imbalanced. A model predicting all negatives would get about:

```
accuracy = 1 - 0.1284 = 0.8716
F1 = 0
```

Therefore, accuracy is not the main metric here. F1, recall, precision, AP, and ROC AUC are more informative.

Results

TFM results, tuned on validation F1:

model	threshold	accuracy	F1	precision	recall	AP	ROC AUC
TabPFN	0.112	0.592	0.276	0.179	0.607	0.245	0.652
TabICL	0.127	0.798	0.542	0.382	0.929	0.483	0.894

KGE results, tuned on validation F1:

model	threshold	accuracy	F1	precision	recall	AP	ROC AUC
ComplEx	0.440	0.628	0.198	0.137	0.357	0.226	0.616
TransR	0.160	0.766	0.215	0.189	0.250	0.194	0.639
TransE	0.086	0.610	0.388	0.243	0.964	0.325	0.826
DistMult	0.230	0.417	0.181	0.110	0.500	0.111	0.426

Interpretation

This is the most important experiment.

In the total-order antisymmetry dataset, TransE and TransR were very strong because the relation had a simple global ranking structure.

In sparse `supervised_by`, that shortcut is removed. The relation is a sparse directed graph with many both-negative unrelated pairs.

The result changes:

```
TabICL becomes the best model by F1, AP, and ROC AUC.
TransE has high recall but low precision.
TransR is weak.
DistMult remains weak, as expected.
```

`TabICL`:

```
F1 = 0.542
recall = 0.929
precision = 0.382
ROC AUC = 0.894
```

This means it is able to rank true `supervised_by` edges above unrelated negatives and reverse negatives much better than the other models.

`TransE`:

```
F1 = 0.388
recall = 0.964
precision = 0.243
ROC AUC = 0.826
```

This means TransE retrieves many positives but overpredicts positives.

`DistMult`:

```
F1 = 0.181
ROC AUC = 0.426
```

This is expected because DistMult has a symmetric score and is poorly matched to directional antisymmetry.

The correct claim is:

```
On realistic sparse antisymmetry, TabICL outperforms the implemented KGE
baselines.
```

This is stronger and more relevant than the total-order antisymmetry result.

Why Threshold Tuning Matters

The sparse `supervised_by` test is imbalanced.

The true positive rate is:

```
0.1284
```

With fixed threshold `0.5`, several models predicted almost no positives. That produced high accuracy but zero or near-zero F1.

Validation F1 tuning selected lower thresholds:

```
TabPFN: 0.112
TabICL: 0.127
TransE: 0.086
TransR: 0.160
DistMult: 0.230
ComplEx: 0.440
```

This is methodologically appropriate because the model scores are not calibrated for a 12.8 percent positive-rate dataset.

The threshold is selected on validation only and then evaluated on test.

Code references:

```
TFM threshold tuning: run_minimal_symmetry_tfm_experiment.py:251
TFM threshold applied: run_minimal_symmetry_tfm_experiment.py:456
KGE threshold tuning: run_kge_relation_baselines.py:197
KGE threshold applied: run_kge_relation_baselines.py:419
```

Main Scientific Conclusions

1. Group-based symmetry is too easy for relational baselines and too structured as a pure symmetry test.

The first symmetry dataset is useful as a sanity check. It shows that the task is solvable. But because it is generated from hidden groups, it also encodes transitivity.

The poor TFM performance there shows that TabPFN/TabICL do not automatically recover the hidden grouping structure from raw categorical pairs.

2. Non-transitive symmetry removes the group/transitivity shortcut, but reverse-holdout remains easy for symmetric KGE scoring.

The non-transitive generator successfully creates symmetric but non-transitive relations.

However, under reverse-holdout, every test row is the reverse of a train row. Symmetric scoring models such as DistMult, and near-symmetric TransE/TransR configurations, can solve this directly.

This is a useful positive-control setup, not a proof of abstract symbolic symmetry.

3. Total-order antisymmetry is not the same as realistic `supervised_by`.

The seniority dataset is antisymmetric, but it is a total order. TransE and TransR solve it because they can learn a geometric ranking direction.

This explains why TransE/TransR are strong there despite literature showing TransE can struggle with arbitrary antisymmetric relations.

4. Sparse `supervised_by` is the strongest current experiment.

Sparse `supervised_by` removes the global-order shortcut and introduces both-negative unrelated pairs.

This is closer to the relation originally intended:

```
some student-supervisor pairs are true
reverse is false
most unrelated pairs are false in both directions
```

In this setting, `TabICL` is the best model among those tested.

What The Code Does Not Yet Prove

The current experiments use the same entity universe across train, validation, and test.

Therefore, the results do not yet prove:

```
generalization to completely new entities
generalization to completely new relation matrices
learning of an abstract symbolic rule independent of entity identity
```

They test:

```
generalization from observed pair evidence to held-out rows within the same
```

```
entity universe
```

This is still meaningful, especially for sparse `supervised_by`, but the claim should be stated carefully.

Recommended Presentation Narrative

We started with a simple symmetry dataset generated by hidden groups. KGE models solved it, but TFMs did not. We then noticed that this dataset was not a pure symmetry task because group membership also induces transitivity.

We therefore generated a non-transitive symmetric dataset using random undirected graph edges. This removed the transitivity shortcut. KGE models still solved reverse-holdout symmetry because the test row is the reversed version of a train row, which matches their scoring bias.

For antisymmetry, the first dataset used hidden seniority rank. This was a total-order relation, and TransE/TransR solved it very well. This was not surprising after inspection because translation models can encode a global order.

Finally, we implemented a sparse supervised_by dataset. This is closer to the intended real-world relation: only some student-supervisor pairs are positive, reverse directions are negative, and unrelated pairs are negative both ways. In this harder and more realistic setting, TabICL outperformed all implemented KGE baselines by F1, AP, and ROC AUC.

Files To Show During The Presentation

Dataset generators:

```
generate_minimal_symmetry_dataset.py
generate_nontransitive_symmetry_dataset.py
generate_minimal_antisymmetry_dataset.py
generate_sparse_supervised_by_dataset.py
```

Model runners:

```
run_minimal_symmetry_tfm_experiment.py
run_kge_relation_baselines.py
```

Most relevant result files:

```
results_symmetry_tfms/metrics.csv
results_symmetry_kge/metrics.csv
results_non_transitive_symmetry_kge/metrics.csv
results_antisymmetry_tfms/metrics.csv
results_antisymmetry_kge/metrics.csv
```

```
results_sparse_supervised_by_tfms/metrics.csv
results_sparse_kge/metrics.csv
```

Most relevant final claim:

```
TabICL is the strongest model on sparse supervised_by antisymmetry, the
most realistic relation dataset built so far.
```